CSC 316

Handout: Interaction & Events

What this is. How a user action — a click, a hover, a dropdown change — becomes a change on screen. Builds directly on runes: events set `$state`, and the rest follows. **Companion to Lab 4.**

> **See also:** Svelte 5 Runes & Reactivity — interaction is just "an event updates `$state`," so read that first.

Grounded in the **Billboard dashboard** from Lab 4: a `KpiRow`, a `Timeline`, a `GenreBars` chart and a `Scatter`, all answering to one selection. Lab 4 builds two interactions on it — **hover a genre to highlight** (`hoveredGenre`) and **brush a span of years to filter** (`yearRange`) — and both are the same move: an event writes one `$state`, everything else is `$derived`.

---

1. Event handlers

An event handler is an attribute holding a function. Svelte 5 uses plain lowercase names — the same ones the browser uses:

```
<button onclick={() => (hoveredGenre = 'Pop')}>Show Pop</button>

<circle
  cx={xScale(d.Danceability)}
  cy={yScale(d.Energy)}
  onmouseenter={() => (hovered = d)}
  onmouseleave={() => (hovered = null)}
/>
```

> **Column names are the file's, not tidied.** The Billboard CSV ships `Song`, `Artist`, `BPM`, `Energy`, `Danceability`, `Weeks at Number One` — capitalized, and one with spaces (`d['Weeks at Number One']`). The only two lowercase fields below are ones **you** derive in Lab 4: `year` (from the two-digit `Date`) and `genre` (the text before the first `;` in `CDR Genre`, blank → `"Unlabeled"`).

The ones you'll actually use:

Handler	Fires when
`onclick`	The element is clicked
`onmouseenter` / `onmouseleave`	The pointer enters / leaves it
`onmousemove`	The pointer moves over it (for tooltip position)
`onchange`	A `<select>` or `<input>` commits a new value
`oninput`	A field or slider changes *as you drag*
`onkeydown`	A key is pressed (Escape to clear a selection)

The handler is just a function, so both of these work:

```
<button onclick={() => (selected = null)}>Clear</button>   <!-- inline -->
<button onclick={clearSelection}>Clear</button>            <!-- named -->
```

Write `onclick={clearSelection()}` with parentheses and you've called it immediately and passed its *result* as the handler — a genuinely common bug that shows up as "the button fires once on page load and never again."

The event object. Handlers receive an event with details — where the pointer is, which key was pressed:

```
<svg onmousemove={(e) => { mouseX = e.offsetX; mouseY = e.offsetY; }}>
```

`e.offsetX/Y` are relative to the element — usually what you want for positioning a tooltip. `e.clientX/Y` are relative to the window.

> **Svelte 5 changed this syntax.** Older examples (and a lot of what an LLM has read) use `on:click` with a colon. That's Svelte 4. In Svelte 5 it's `onclick`. If Claude writes `on:click`, it has drifted to the old idiom — re-prompt, and put the rule in your `CLAUDE.md`.

---

2. `bind:` — two-way binding

For form controls, wiring an `onchange` handler by hand is busywork. `bind:` does both directions at once — the control shows the state, and changing the control updates the state:

```
<script>
  let selectedGenre = $state('All');
</script>

<select bind:value={selectedGenre}>
  <option>All</option>
  {#each genres as g}<option>{g}</option>{/each}
</select>
```

No handler, no `e.target.value`. Pick an option and `selectedGenre` *is* the new value.

> **Lab 4's dashboard has no dropdown** — its two interactions are a hover and a brush (§3). The `<select>` you've actually built is Lab 2's `TerminalFilter`, and a genre dropdown is a natural thing to *add* here. Both work the same way, which is the point.

Common bindings:

```
<input type="range" min="1958" max="2025" bind:value={year} />   <!-- slider -->
<input type="checkbox" bind:checked={showLabels} />              <!-- toggle -->
<input type="text" bind:value={query} />                         <!-- search box -->
```

Numbers need care. An `<input>`'s value is a **string**, so `bind:value` on a range slider gives you `"1994"`. Svelte converts automatically for `type="range"` and `type="number"`, but for a plain text input you'll want `+query` before doing arithmetic — the same string-versus-number issue as CSV parsing.

Binding across components. When the dropdown lives in its own component — Lab 2's `TerminalFilter` — the child declares the prop as `$bindable` so the parent's state can flow both ways:

```
<!-- src/lib/TerminalFilter.svelte -->
<script>
  let { selected = $bindable(), terminals } = $props();
</script>
<select bind:value={selected}> … </select>
```

```
<!-- +page.svelte -->
<TerminalFilter {terminals} bind:selected={selectedTerminal} />
```

This is the one sanctioned exception to “props flow down.” Use it for controls, and only for controls — a chart component should never write back to the page’s state.

> **A control can also report up with a plain callback prop**, which is what Lab 4’s `Timeline` does with its brush: `<Timeline onbrush={r => (yearRange = r)} />`. Use `bind:` when the child needs to *display* the current value too (a `<select>` shows the selection); use a callback when the child only *emits* (a brush reports an extent). §5 works the brush through in full.

---

3. Driving state from the UI

Here’s the whole pattern, and it’s the reason the labs keep repeating “one state, everything derived”:

> **An event sets one `$state` value. Every view is `$derived` from it. Nothing else updates anything.**

This is Lab 4’s `+page.svelte`, reduced to its skeleton:

```
<script>
  let yearRange = $state(null);                        // ← the ONE selection
                                                       //   [startYear, endYear], or null

  let filtered = $derived(                             // ← everything follows
    yearRange
      ? songs.filter(d => d.year >= yearRange[0] && d.year <= yearRange[1])
      : songs                                          //   null MUST mean "everything"
  );
  let totalSongs = $derived(filtered.length);
  let avgBpm     = $derived(d3.mean(filtered, d => d.BPM));
  let byYear     = $derived(d3.rollups(filtered, v => v.length, d => d.year));
</script>

<Timeline   data={byYear}  onbrush={r => (yearRange = r)} />  <!-- writes it -->
<KpiRow     songs={totalSongs} bpm={avgBpm} />               <!-- reads it -->
<GenreBars  data={filtered} />                               <!-- reads it -->
<Scatter    data={filtered} />                               <!-- reads it -->
```

Brush a decade on the timeline and the cards, the genre bars and the scatter all update — because each is a function of `filtered`, which is a function of `yearRange`. **You never write an update call.**

Why this is a correctness property, not a style preference. Suppose one view reads the full `songs` array instead of `filtered`. It won’t error. It will look fine. It will simply show the wrong thing — a chart that silently contradicts its neighbours while a filter is active. Lab 4 calls this the *frozen view*, and it’s the honesty failure of interactive work: nothing crashes, the chart just quietly lies. **One source of truth is what makes disagreement impossible.**

The same move, one notch gentler: linking. Lab 4's other interaction never filters anything — hovering a genre bar just lights that genre up in the scatter. It's the identical shape, and the derived thing is an *attribute* rather than a dataset:

```
<script>
  let hoveredGenre = $state(null);                      // ← a second, independent selection
</script>

<!-- in GenreBars: the event writes it -->
<rect onmouseenter={() => (hoveredGenre = d.genre)}
      onmouseleave={() => (hoveredGenre = null)} … />

<!-- in Scatter: opacity is derived from it -->
<circle fill-opacity={!hoveredGenre || d.genre === hoveredGenre ? 0.7 : 0.1} … />
```

The bar heights don't move and no row is removed — if dots *vanish* on hover instead of dimming, it filtered when you asked it to highlight. That distinction is §5.

Make the selection clearable. Any interaction that filters needs an obvious way back — clearing the brush, a "Clear" button, an "All" option, clicking the background, Escape:

```
<svg onclick={() => (yearRange = null)}> … </svg>
```

A dashboard a user can get stuck inside is a bug, even if every chart is correct. Note the direction of the `null`: for a brush, **`null` must mean "everything," not "nothing."**

---

4. Hover and tooltips

A tooltip is one more piece of state: **which mark is hovered**, plus where the pointer is.

```
<script>
  let hovered = $state(null);
  let mouse = $state({ x: 0, y: 0 });
</script>

<svg onmousemove={(e) => (mouse = { x: e.offsetX, y: e.offsetY })}>
  {#each songs as d}
    <circle
      cx={xScale(d.Danceability)} cy={yScale(d.Energy)} r="4"
      fill-opacity={hovered && hovered !== d ? 0.15 : 0.7}
      onmouseenter={() => (hovered = d)}
      onmouseleave={() => (hovered = null)}
    />
  {/each}
</svg>

{#if hovered}
  <div class="tooltip" style="left: {mouse.x + 12}px; top: {mouse.y + 12}px">
    <strong>{hovered.Song}</strong><br />
    {hovered.Artist} · {hovered['Weeks at Number One']} weeks at #1
  </div>
{/if}
```

Three things to notice:

- **`hovered` holds the whole row**, not an index. Then the tooltip can read any field.
- **The `{#if}` means no tooltip element exists** when nothing is hovered — nothing to hide or position off-screen.

- **The other marks dim** via `fill-opacity`, derived from `hovered`. Highlighting *is* just another derived attribute.

> **Don't let the tooltip move the page.** Position it `absolute` (or `fixed`) so it floats above the chart. A tooltip in normal document flow pushes content around as it appears, and the chart jumps under the pointer — which feels broken even though it's "working." Add `pointer-events: none` to its CSS so it can't itself swallow the mouse events and cause a flicker loop.

---

5. Coordinating multiple views

Brushing and linking is the pattern where selecting in one view highlights or filters the others. With the single-state model you already have it — a brush is just a control that writes to `$state`, like a dropdown with more pixels.

```
<script>
  let yearRange = $state(null);                         // [1980, 1995] or null

  let filtered = $derived(
    yearRange
      ? songs.filter(d => d.year >= yearRange[0] && d.year <= yearRange[1])
      : songs
  );
</script>

<Timeline  onbrush={r => (yearRange = r)} />   <!-- writes the state -->
<GenreBars data={filtered} />                  <!-- reads it, derived -->
<Scatter   data={filtered} />                  <!-- reads it, derived -->
```

D3's `d3.brushX` is the one place we let D3 touch interaction besides axes, because dragging a selection rectangle is fiddly to build by hand. Same treatment as an axis: attach it to a `<g>`, and have its event **invert the pixel extent back through the x-scale** to a `[startYear, endYear]` before writing your `$state`. The brush gives you pixels; your state stores data.

Link or filter — decide on purpose. Two different interactions, often confused:

	Linking (highlight)	Filtering (remove)
The other views	keep all data, dim the unselected	recompute on the subset only
Good for	seeing where a subset sits *in context*	reading exact values for the subset
Cost	can be visually noisy	loses the context the subset came from

Neither is more correct. What's wrong is doing it by accident — a chart that dims some bars while a neighbour silently recomputes its axis is telling two stories at once.

> **N views, one state — not N² connections.** The reason this scales: with a shared selection, adding a fourth chart means adding one `$derived`. If instead each view updated the others directly, four views would need twelve wires, and every new one would need connecting to all the rest. That's why "one state, everything derived" is the architecture and not just tidiness.

Appendix — Interaction patterns cheatsheet

Each pattern, the state it needs, and what to watch for.

Pattern	State	Views read	Watch out for
Dropdown filter	`selected` (string)	`$derived` filtered data	Include an “All” option so it can be cleared
Toggle	`showX` (boolean)	conditional rendering	`bind:checked`, not `bind:value`
Slider / year scrub	`year` (number)	data filtered to that value	`<input type="range">` gives a string on plain inputs
Hover detail	`hovered` (the row, or `null`)	tooltip + dimmed marks	`pointer-events: none` on the tooltip
Click to select	`selected` (the row, or `null`)	highlight + a detail panel	Provide a way to deselect (click background, Escape)
Brush a range *(Lab 4)*	`yearRange` (`[min, max]` or `null`)	every view derives from it	`null` must mean “everything”, not “nothing”; invert pixels → data before storing
Hover a category to link *(Lab 4)*	`hoveredGenre` (string or `null`)	marks dim via `fill-opacity`	Cheapest useful interaction — one state, one attribute. Counts must **not** move
Cross-filter	one state per control	all views derive from all of them	Every view must read the *same* filtered set

The checklist for any interaction you add:

1. Can you point at the **single** `$state` it writes?
2. Does **every** view derive from it — none reading the global data?
3. Is there an obvious way to **clear** it?
4. Can a first-time user **tell it’s interactive** (cursor change, hover feedback, a hint)?
5. Did the previous interaction **still work** after you added this one — and is there a green commit proving it?